Variáveis Compostas
Tuplas, Listas

UFSC / CTC / INE
Professora: Luciana Rech

INE5402 - PROGRAMAÇÃO ORIENTADA A OBJETOS I
INE5603 – INTRODUÇÃO A PROGRAMAÇÃO ORIENTADA A OBJETOS

Introdução

- Até o momento, os programas que escrevemos manipularam os tipos real, inteiro, literal, lógico.
  - Um elemento (variável) declarado como real, por exemplo, constitui-se em uma posição de memória que permite o armazenamento de exatamente um valor, e do tipo real.
  - Como elementos desse tipo armazenam apenas um valor, são denominadas **variáveis simples**.

Introdução

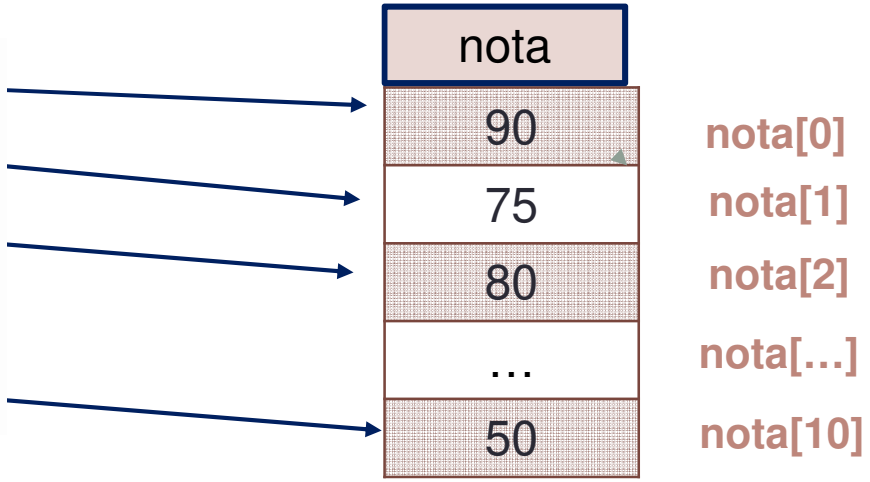
- Se, em um programa, necessitarmos manipular a nota de um aluno, sabemos que esta constitui-se em um único valor e, portanto, uma variável do tipo real, por exemplo, pode ser usada para armazenar essa nota.
  - Como fazer, entretanto, se necessitarmos armazenar as várias notas que o aluno obteve em uma disciplina?
  - Para resolver esse problema, as linguagens de programação apresentam os tipos denominados **estruturados**.
    - Um elemento que armazena mais um valor é denominado de variável composta.

Exemplo: Como armazenar as notas de um aluno utilizando:

variáveis simples

```
nota_1 = int(input("Digite a nota 1:"))
nota_2 = int(input("Digite a nota 2:"))
nota_3 = int(input("Digite a nota 3:"))
...
nota_10 = int(input("Digite a nota 10:"))
```

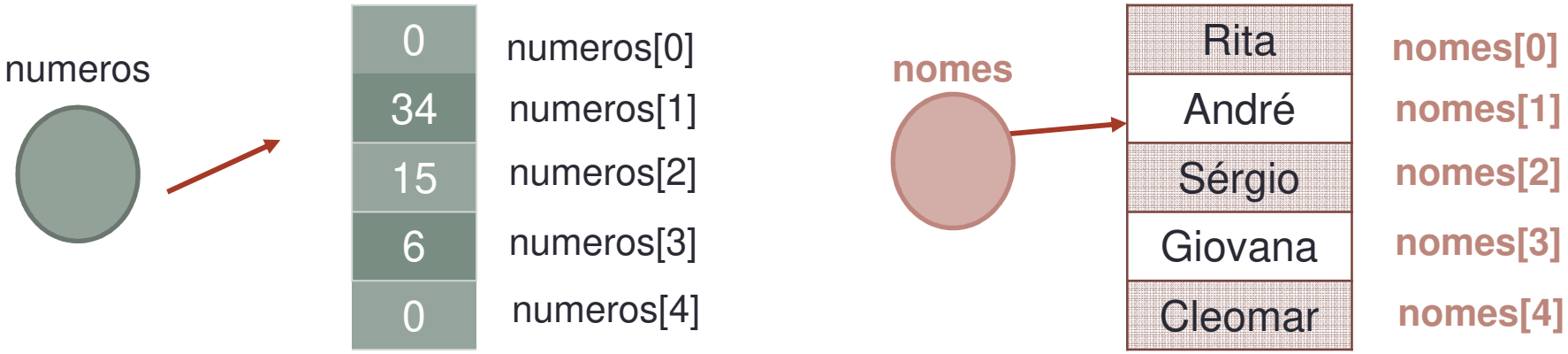
variável composta



Neste exemplo, utilizando variáveis simples seriam necessárias 10 variáveis. Utilizando variável composta (list) será necessário apenas uma única estrutura onde serão armazenadas todas as notas, neste caso as informações ficam organizadas por índices.

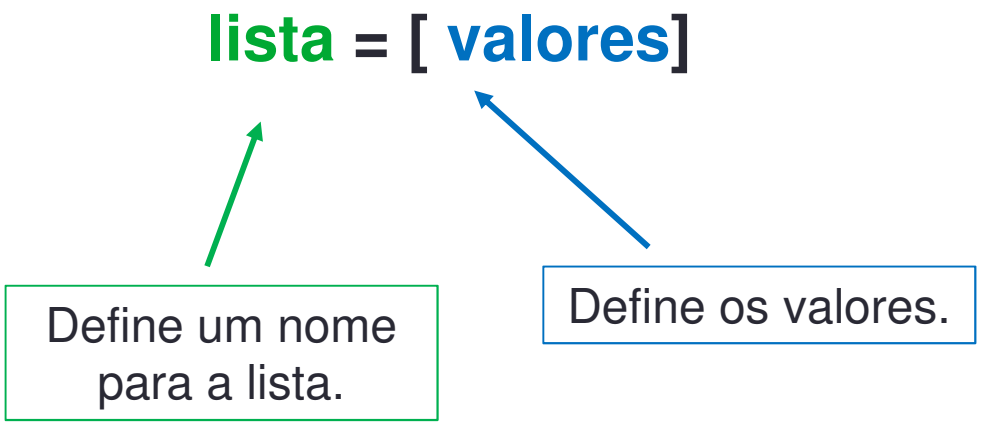
Introdução

- Usando estrutura composta podemos armazenar e manipular facilmente uma coleção de valores de determinado tipo ou mesmo uma coleção de objetos.



- Em Python, podemos utilizar estrutura composta através de tupla, lista e dicionário.

- **Declaração**:



- **Exemplos de declaração e inicialização**:

valores_aleatorios = [12.3, 5, “joao”]

nomes = [“ray”, “peter”, “fritz”]

numeros = list() numeros = [ ]

- **Armazenamento**:

  numeros[0] = 14

  numeros[2] = 78

  nomes[1] = “Ana Maria”
  nomes[6] = “Luiza”

- **Recuperação de um dado:**

  n = numeros[1]

  nome = nomes[4]

- O Python oferece 2 estruturas de dados bem parecidas:
  - Listas
    - Lista_Exemplo = [valor1, valor2, valor3]
  - Tuplas
    - Tupla_Exemplo = (valor1, valor2, valor3)

- Qual é a diferença?
  - Tuplas são immutáveis!!!

Não armazenam novos valores ou atualizam o conteúdo de valores existentes em tempo de execução, ou seja, se for necessário alterar algum valor, isso deve ser feito no código fonte e o programa deverá ser compilado novamente.

Quando usar tuplas:

Você deve usar uma tupla sempre que tiver valores, itens, que nunca serão mudados, serão constantes.

Por exemplo, se você for criar um programa que use coordenadas geográficas, cada ponto da terra pode ser representado por uma tupla, e este valor jamais deverá ser alterado.

Em um cadastro, sobre dados que não serão alterados, por exemplo: ID, CPF, ... podem ser representados por uma tupla.

Duas vantagens das tuplas em relação a listas: são mais rápidas e protegem seus scripts de sofrerem alterações.

Como criar uma tupla:

```
tuplaNomes = (“Ana”, “Pedro”)
```



parênteses para declarar tuplas ( )

Como criar a lista:

- Exemplos:
- lista_exemplo = [1, 2, 3, “peter”, 1.5, “fox”, 4]



colchetes para declarar listas[ ]

- lista = list(range(10))
  - [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
  - # neste exemplo, range indica o tamanho da lista que está sendo criada (tamanho = 10, com índices de 0 a 9)
- lista = [ ] ou lista = list()
  - # criando um lista vazia (os elementos serão inseridos em tempo de execução do programa)

Como manipular variáveis compostas (listas):

- - Usando laço **for** para preencher a lista:

1) lista = list()

```
    for i in range (0, 10):
        lista.append(input("Digite um valor:" ))
```

2) lista = list(range(5))

```
    for i  in   range (0, len(lista)):
        lista[i] = i+10
```

- - *Append vs Insert*
  - *Append adiciona um elemento no fim da lista*
  - *seuArray.append("peter") // Adiciona o valor "peter" no fim da lista*

  - Insert insere na posicao
  - *seuArray.insert(1, "peter") //Adiciona o valor "peter" na posicao 1 da lista (organiza automaticamente as outras chaves)*

Apagando valores

Tupla

Somente a tupla completa:
- del nome_da_tupla

Lista

Apagando um elemento pela sua posição (índice):
- del nome_da_lista[pos]
- nome_da_lista.pop(pos)

Apagando um elemento pelo seu conteúdo:
nome_da_lista.remove(‘conteúdo’)

```python
teste = [1,2,3,4,'ana','pedro',14.5]
print(teste)
del teste[3]
print(teste)
del teste

teste1 = [1,2,3,4,5,6]
teste1.pop(2)
print(teste1)
teste1.remove(5)
print(teste1)

if 4 in teste1:
    teste1.remove(4)
print(teste1)
```

Saída:

```
[1, 2, 3, 4, 'ana', 'pedro', 14.5]
[1, 2, 3, 'ana', 'pedro', 14.5]
[1, 2, 4, 5, 6]
[1, 2, 4, 6]
[1, 2, 6]
```

Exemplo:

```
lista = []
sum = 0

print(f"Escreva o tamanho da lista:")
size = int(input())

for i in range(0, size):
    print(f"Escreva um numero:")
    lista.append(int(input()))

print(lista)

for cont in range(0, len(lista)):
    sum += lista[cont]

print(f"A soma e: {sum}")
```

```
Escreva o tamanho da lista:
3
Escreva um numero:
1
Escreva um numero:
4
Escreva um numero:
6
[1, 4, 6]
A soma e: 11
```

Imprimindo uma tupla ou lista

```
lanche = ('pizza','suco','pudim','bolo')
print(lanche)
print(lanche[1:])   #imprime os elementos da posição 1 até o final da tupla
print(lanche[-1])   #imprime o último elemento
print(lanche[-2])   #imprime o penúltimo elemento
print(lanche[1:2])  #imprime os elementos entre as posições 1 e anterior a 2
print(lanche[1:3])  #imprime os elementos entre as posições 1 e anterior a 3
```

Saída:

```
('pizza', 'suco', 'pudim', 'bolo')
('suco', 'pudim', 'bolo')
bolo
pudim
('suco',)
('suco', 'pudim')
```

Imprimindo uma tupla ou lista
(apenas o conteúdo)

```
for comida in lanche:
    print(f'Eu vou comer: {comida}')
# ou
for cont in range(0, len(lanche)):
    print(f'\nEu vou comer: {lanche[cont]}')
```

```
Eu vou comer: pizza
Eu vou comer: suco
Eu vou comer: pudim
Eu vou comer: bolo

Eu vou comer: pizza

Eu vou comer: suco

Eu vou comer: pudim

Eu vou comer: bolo
```

Imprimindo uma tupla ou lista
(enumerate permite mostrar a posição e o conteúdo)

```
for pos,comida in enumerate(lanche):
    print(f'Eu vou comer: {comida} na posição {pos}')
```

```
Eu vou comer: pizza na posição 0
Eu vou comer: suco na posição 1
Eu vou comer: pudim na posição 2
Eu vou comer: bolo na posição 3
```

```
1  lista = ['maçã','uva', 'banana', 'pêssego']
2
3  for pos,valor in enumerate(lista):
4      print(f'{pos} - {valor}')
```

```
>>> %Run Exemplo_enumerate.py
 0 - maçã
 1 - uva
 2 - banana
 3 - pêssego
```

```python
nomes = ["Pedro", "Bruno", "Carlos"]
for c in range(0,len(nomes)):
    print(f'Nome: {nomes[c]}, na posição: {c}')
```

Saída:

```
Nome: Pedro, na posição: 0
Nome: Bruno, na posição: 1
Nome: Carlos, na posição: 2
```

Imprimindo uma tupla ou lista
“sorted” = ordena apenas para impressão

lanche = (‘pizza’, ‘suco’, ‘pudim’, ‘bolo’)

```
print(sorted(lanche))
print(lanche)
```

Saída:

```
['bolo', 'pizza', 'pudim', 'suco']
('pizza', 'suco', 'pudim', 'bolo')
```

Se você deseja ordenar (de forma definitiva) a lista, utilize a função sort():

```
1  nome = list()
2  print(nome)
3  for i in range (0,3):
4      nome.append(str(input("Digite um nome: ")))
5  print(nome)
6  print(sorted(nome))
7  print(nome)
8  nome.sort()
9  print(nome)
```

Saída:

```
[]
Digite um nome: José
Digite um nome: Débora
Digite um nome: Carolina
['José', 'Débora', 'Carolina']
['Carolina', 'Débora', 'José']
['José', 'Débora', 'Carolina']
['Carolina', 'Débora', 'José']
```

```python
#uso count e index
a = (1,2,3)
b = (4,5)
#concatenando tuplas
c = a + b
print(c)

#para saber quantas vezes um determinado elemento aparece na tupla
print(f'Número de vezes que o elemento 5 aparece na tupla',c.count(5))

#para descobrir a posição de determinado elemento na tupla
print(f'Posição do valor 4 na tupla: {c.index(4)}')
```

Saída:

```
(1, 2, 3, 4, 5)
Número de vezes que o elemento 5 aparece na tupla 1
Posição do valor 4 na tupla: 3
```

Atenção
Ligação entre listas

```
a = [1,2,3,4,5]
b = a
print(a)
print(b)

b[3]=12
print(f'Lista B: {b}')
print(f'Lista A: {a}')
```

Saída:

```
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5]
Lista B: [1, 2, 3, 12, 5]
Lista A: [1, 2, 3, 12, 5]
```

Copiando uma lista

```
lista_a = [1,2,3,4]
lista_b = [5,6,7,8]
c = lista_a[:]  #c é uma cópia da lista_a  (sem ficar vínculo)
c[2] = 56
print('Lista A',lista_a)
print('Lista C',c)
```

```
Lista A [1, 2, 3, 4]
Lista C [1, 2, 56, 4]
```